

Software Architecture

EE 8217/EC 8208



Based on Software Architecture in Practice (3rd Edition)
By Len Bass, Paul Clements, Rick Kazman

Outline

Introduction to Software Architecture

- What is Software Architecture?
- Architectural Structures and Views
- Categories of Architectural Structures
- Architectural Patterns

What is Software Architecture?

The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.

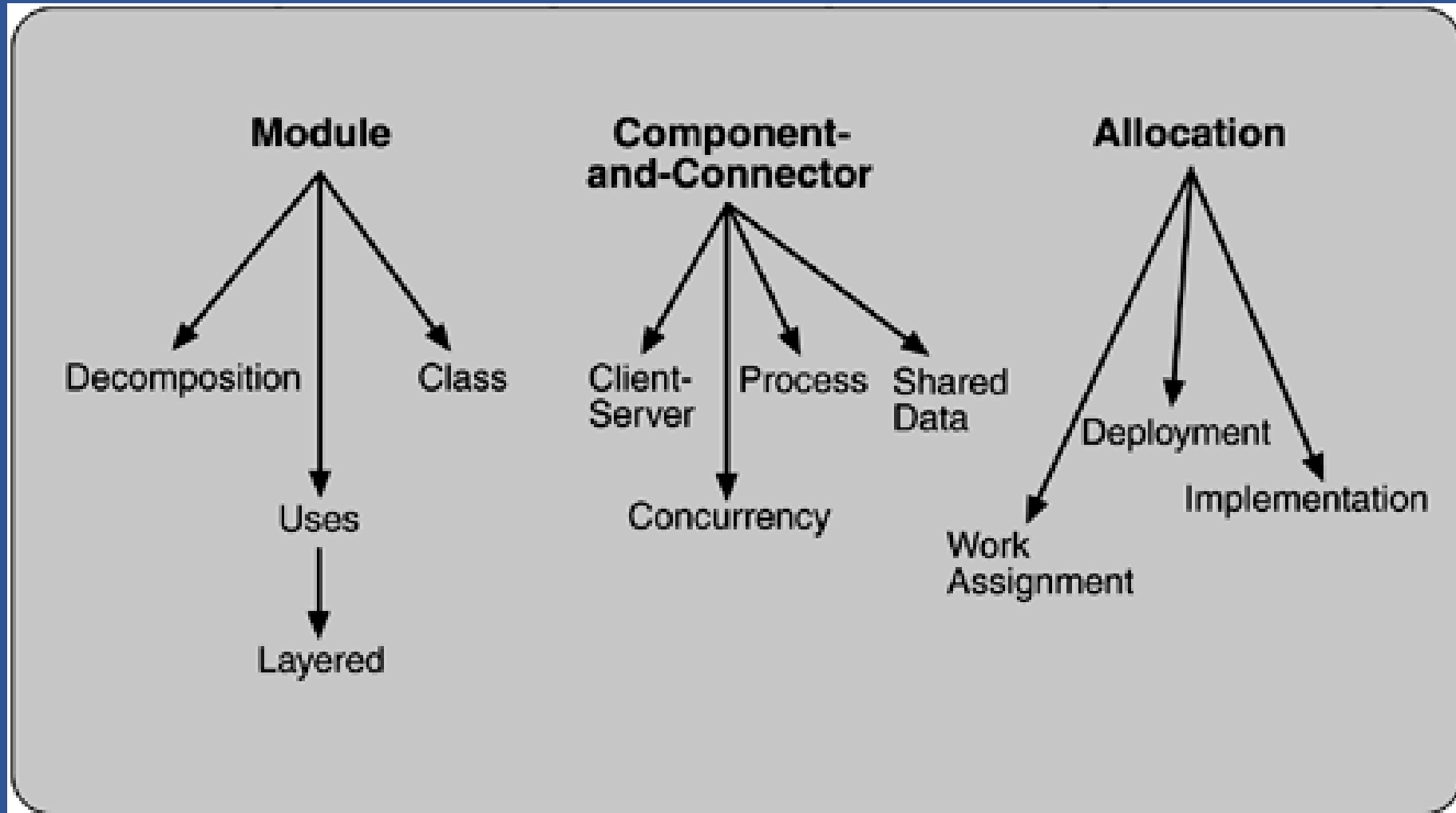


Winchester Mystery House???



- Although it is often discussed as a “mystery house,” it is also an interesting example of:
 - Continuous architectural modification
 - Lack of an unified architectural plan
 - Complex structural evolution over time
- Systems developed without proper architecture can become difficult to understand, maintain, and extend similar to the confusing structure of the Winchester Mystery House.

Architecture Is a Set of Software Structures

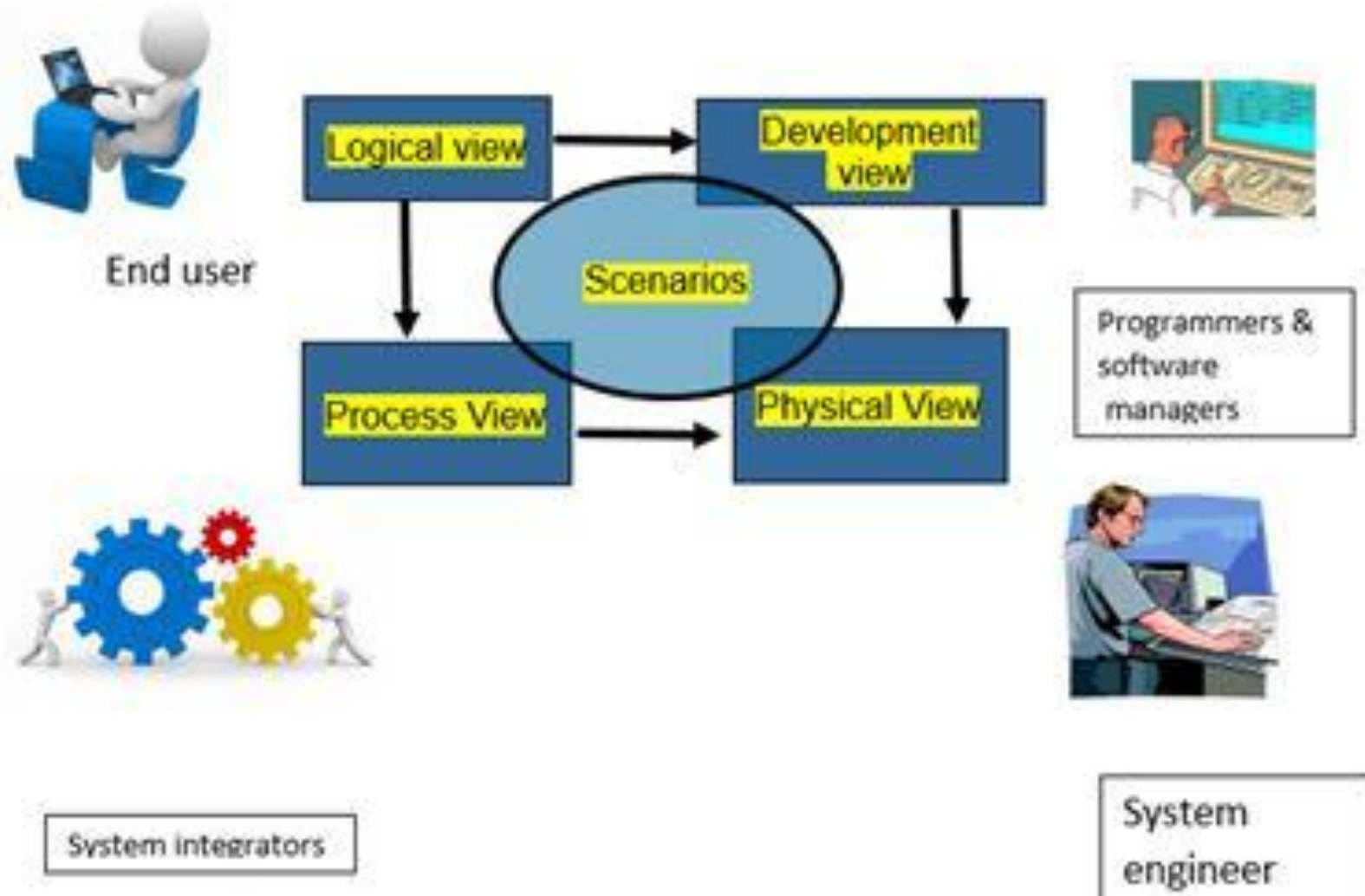


Several different views of the human body: the skeletal, the vascular, and the X-ray.



Architectural Structures and Views

- A view is a representation of a coherent set of architectural elements, as written by and read by system stakeholders.
- It consists of a representation of a set of elements and the relations among them.
- A structure is the set of elements itself, as they exist in software or hardware.
- In short, a view is a representation of a structure



Categories of Architectural Structures

1) Modules Structures

- Static structures
- Focus on the way the system's functionality is divided up and assigned to implementation teams.
- **Examples:**
 - Layered structure (UI, Business Logic, Data)
 - Package diagrams

2) Component-and-Connector (C&C) Structures

- Dynamic structures
- focus on the way the elements interact with each other at runtime to carry out the system's functions
- **Examples:**
 - Client–Server systems
 - Microservices communication
 - Event-driven systems

3) Allocation Structures

- Describes the mapping from software structures to the system's organizational, developmental, installation, and execution environments.
- Maps software elements to:
 - Hardware
 - Deployment environments
 - Development teams
- **Examples:**
 - Deployment diagrams
 - Mapping modules to teams
 - Cloud vs on-premise setup

TABLE 1.1 Useful Architectural Structures

	Software Structure	Element Types	Relations	Useful For	Quality Attributes Affected
Module Structures	Decomposition	Module	Is a submodule of	Resource allocation and project structuring and planning; information hiding, encapsulation; configuration control	Modifiability
	Uses	Module	Uses (i.e., requires the correct presence of)	Engineering subsets, engineering extensions	"Subsetability," extensibility
	Layers	Layer	Requires the correct presence of, uses the services of, provides abstraction to	Incremental development, implementing systems on top of "virtual machines"	Portability
	Class	Class, object	Is an instance of, shares access methods of	In object-oriented design systems, factoring out commonality; planning extensions of functionality	Modifiability, extensibility
	Data model	Data entity	{one, many}-to-{one, many}, generalizes, specializes	Engineering global data structures for consistency and performance	Modifiability, performance
C&C Structures	Service	Service, ESB, registry, others	Runs concurrently with, may run concurrently with, excludes, precedes, etc.	Scheduling analysis, performance analysis	Interoperability, modifiability
	Concurrency	Processes, threads	Can run in parallel	Identifying locations where resource contention exists, or where threads may fork, join, be created, or be killed	Performance, availability
Allocation Structures	Deployment	Components, hardware elements	Allocated to, migrates to	Performance, availability, security analysis	Performance, security, availability
	Implementation	Modules, file structure	Stored in	Configuration control, integration, test activities	Development efficiency
	Work assignment	Modules, organizational units	Assigned to	Project management, best use of expertise and available resources, management of commonality	Development efficiency

fewer is better

- The larger the system, the more dramatic the difference between these structures tends to be; but for small systems we can often get by with fewer.

Which Structures to choose?

- Embedded/IoT Systems
- Web Applications



Too many structures increase complexity

- Wastes timeIncreases
- Documentation effort
- Makes architecture harder to understand

Architectural documentation should produce:

- Better understanding
- Easier maintenance
- Reduced development cost

Architectural Patterns

- Architectural elements are composed in ways that solve particular problems.
- Architectural patterns are **reusable solutions** to common software architecture problems.
- They are created by combining architectural elements in effective ways.
- These solutions have been:
 - Used successfully over time
 - Applied across many different domains

Example:

Layered Pattern

Organize system into layers

Client–Server Pattern

Clients request services from servers

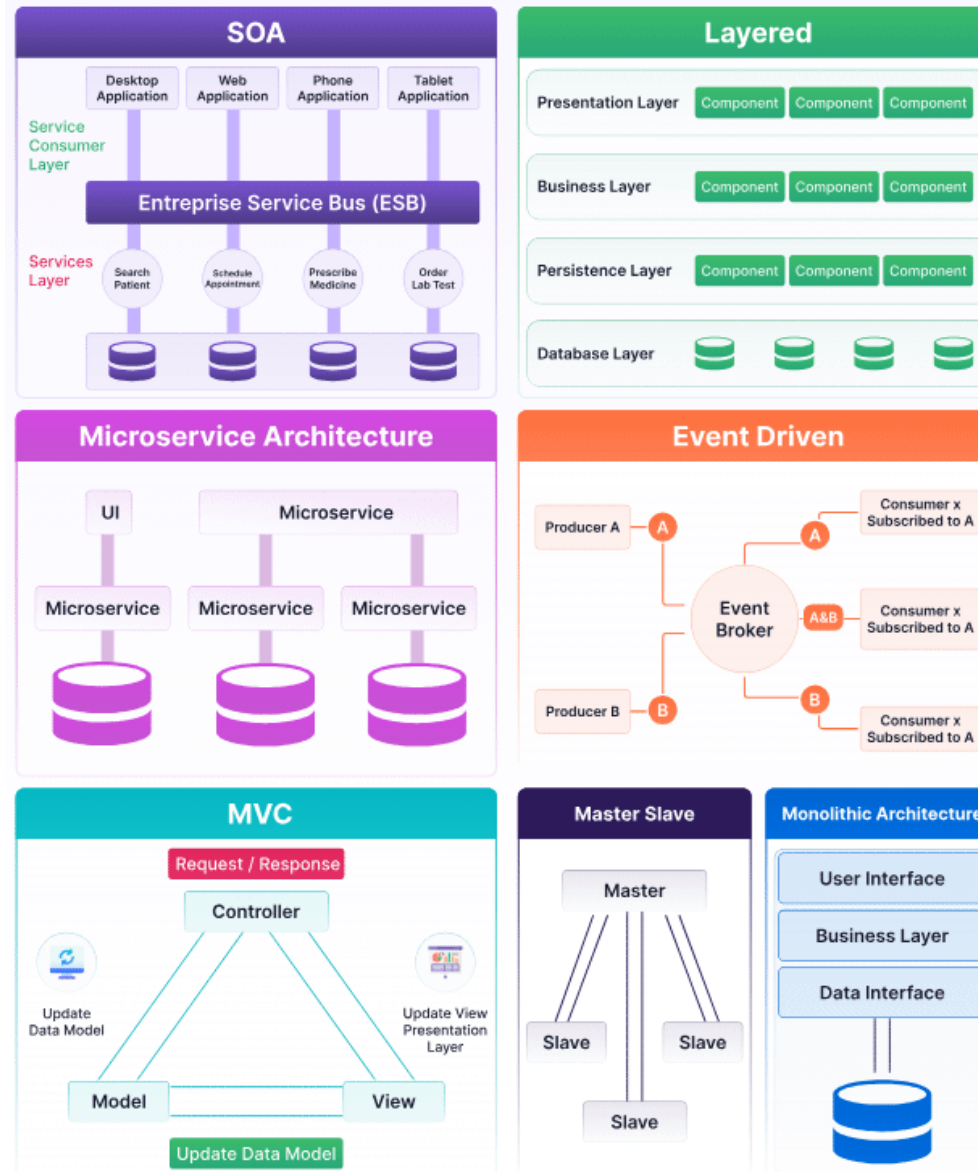
MVC Pattern

Separate data, UI, and control logic

Microservices Pattern

Build system as small independent services

7 ARCHITECTURAL PATTERNS



What Makes a “Good” architecture?

- There is no universally good or bad software architecture.
- An architecture is considered:
More or less suitable depending on the system’s purpose and requirements.
- Architecture Must Match System Goals.
- Examples:
 1. Highly modifiable architecture → good for long-term systems
 2. Simple architecture → better for temporary prototypes

If important design principles are ignored:

- It does not always mean failure
- But it should raise concerns and be investigated

Example Warning Signs:

- Tight coupling between modules
- No modular structure
- Poor scalability
- Difficult maintenance

A good software architecture is not universally "perfect"; it is one that best satisfies the specific goals, constraints, and requirements of a system.

Group Activity

1. Select a Published Architecture
(published software architecture diagram).

Ex: Netflix, Youtube, Uber, Amazon etc.

2. Identify the Architecture Structures

3. What important structures are NOT shown? If any.

4. Discuss what can be analyzed from the architecture, such as:

- ✓ Scalability?
- ✓ Performance?
- ✓ Security?
- ✓ Maintainability?

